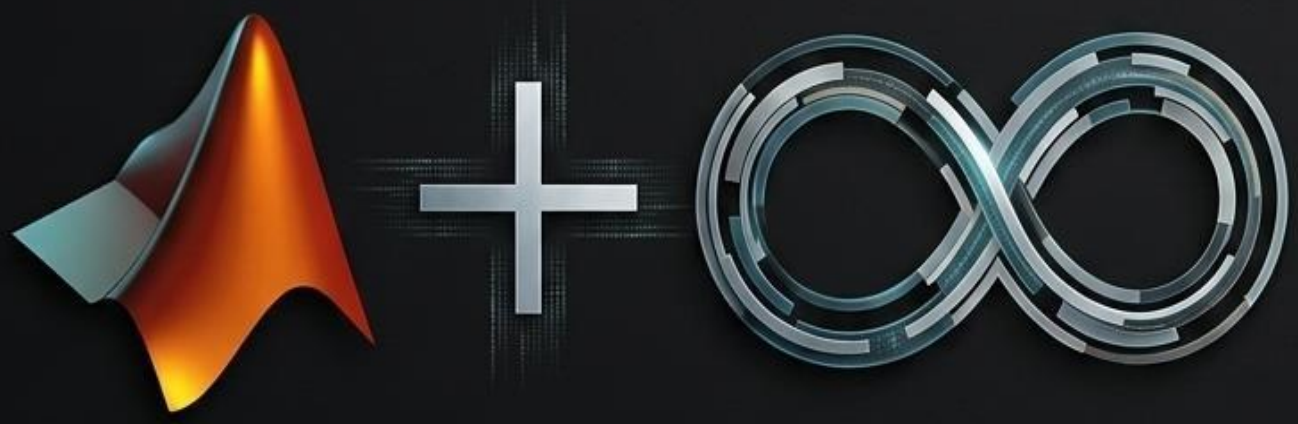
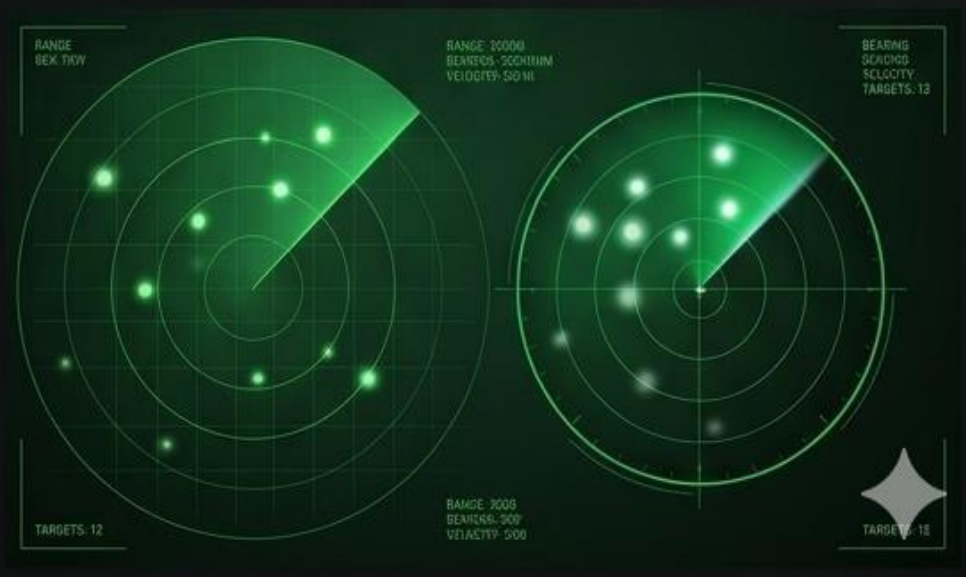
Arduino + Matlab Radar System



Written by:Mo'men Mahmoud Mustafa



1. Introduction

This project presents the design and implementation of a radar system based on an Arduino microcontroller integrated with MATLAB for real-time visualization. The system uses an ultrasonic sensor mounted on a servo motor to detect objects within a defined angular range and display the results graphically using MATLAB.

2. Hardware Components

- Arduino Board
- Servo Motor
- Ultrasonic Sensor (HC-SR04)
- Jumper Wires and Breadboard
- USB Cable for Arduino-MATLAB Communication

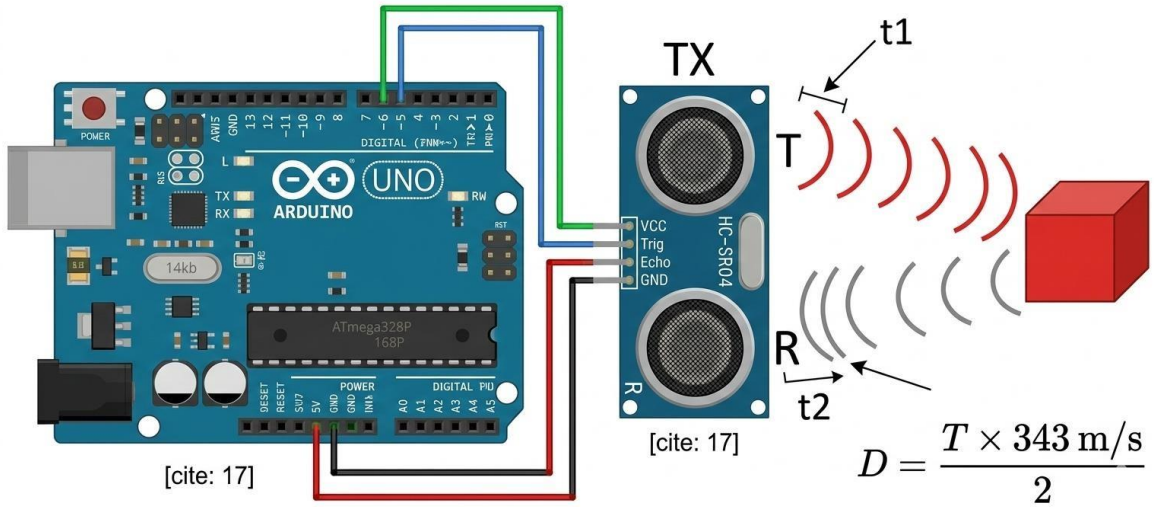


Figure -1 the Ultrasonic configuration

3. Ultrasonic Sensor Working Principle

The ultrasonic sensor consists of four main pins: VCC, Echo, Trigger, and GND. The Trigger pin sends a short 10 microsecond pulse which causes the transmitter (ceramic transducer) to emit ultrasonic sound waves. These waves travel through the air until they hit an object and reflect back to the receiver (ceramic receiver).

The Echo pin outputs a pulse whose duration corresponds to the time taken for the sound wave to travel to the object and return. Since the signal travels forward and backward, the measured time is divided by two to calculate the one-way distance.

Distance is calculated using the formula:

$$Distance = \frac{(Time \times Speed\ of\ Sound)}{2}$$

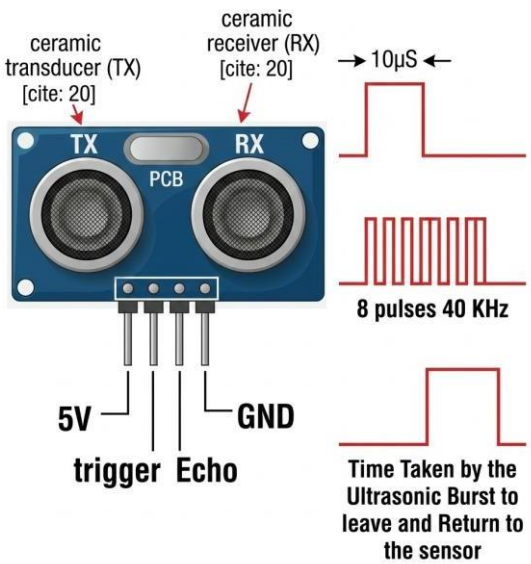


Figure-2 The Ultrasonic concipt

4. Servo Motor Integration

The ultrasonic sensor is mounted on a servo motor to allow scanning across angles from 0° to 180°. The Arduino controls the servo position, enabling the radar system to sweep and detect objects within a semicircular area.

5. Software Implementation

5.1 Arduino Code

The Arduino program controls the servo motor and continuously reads distance measurements from the ultrasonic sensor. The measured values are transmitted to MATLAB through serial communication at 9600 baud rate.

Arduino C

```
#include <Servo.h>
#include <NewPing.h>
#define TRIGGER_PIN  12
#define ECHO_PIN     11
#define MAX_DISTANCE 200   // Maximum distance in cm
NewPing sonar(TRIGGER_PIN, ECHO_PIN, MAX_DISTANCE);
Servo myServo;
int angle;
int samples = 10;   // Number of readings for averaging

void setup() {
  myServo.attach(9);        // Attach servo to pin 9
  Serial.begin(9600);       // Start serial communication
  delay(3000);              // Stabilization delay
}

void loop() {

  for (angle = 0; angle < 180; angle++) {

    myServo.write(angle);   // Move servo to current angle
    delay(30);              // Allow servo to reach position

    long totalDistance = 0;

    // Take multiple readings for better accuracy
    for (int i = 0; i < samples; i++) {
      unsigned int duration = sonar.ping();
      int distance = duration / US_ROUNDTRIP_CM;  // Convert to cm
      totalDistance += distance;
      delay(30);
    }

    int averageDistance = totalDistance / samples;

    // Send angle and distance to MATLAB
    Serial.print(angle);
```

```
  Serial.print(",");
  Serial.println(averageDistance);
 }
}
```

Key functions include:

- Servo motor angle control (0° to 180°)
- Ultrasonic pulse generation and echo reading
- Distance averaging for improved accuracy
- Serial data transmission

5.2 MATLAB Code

MATLAB is used to create a radar-style graphical interface. The program:

- Draws a semicircular radar grid
- Reads real-time data from the serial port
- Converts polar coordinates to Cartesian coordinates
- Displays detected objects dynamically in red.

Matlab

```
clc;
clear;
close all;
% Create Full Screen Figure
figure('units','normalized','outerposition',[0 0 1 1]);
set(gcf,'color','black');
hold on;
% Draw Radar Range Circles
theta = linspace(0, pi, 1000);
ranges = 10:10:100;
for r = ranges
   x = r * cos(theta);
   y = r * sin(theta);
   plot(x, y, 'Color', [0.6 0.8 0.2], 'LineWidth', 1);
end
% Draw Radar Angle Lines
angles = 0:30:180;
for ang = angles
   [x, y] = pol2cart(deg2rad(ang), 100);
   plot([0 x], [0 y], 'Color', [0.6 0.8 0.2], 'LineWidth', 2);
end

% Pre-create scanning lines
for i = 1:180
   [x, y] = pol2cart(deg2rad(i), 100);
   h(i) = plot([0 x], [0 y], 'g', 'LineWidth', 1);
end

% Serial Port Configuration
s = serial('COM4');   % Change COM port if needed
s.BaudRate = 9600;
fopen(s);

% Real-Time Radar Display
```

```
while true
   data = fscanf(s);

   values = strsplit(strtrim(data), ',');

   if length(values) == 2
      angle = str2double(values{1});
      distance = str2double(values{2});

      if angle > 0 && angle <= 180

         % Convert polar to cartesian
         [x_end, y_end] = pol2cart(deg2rad(angle), distance);
         [x_max, y_max] = pol2cart(deg2rad(angle), 100);

         % Update scanning line
         set(h(angle), 'XData', [0 x_end], 'YData', [0 y_end], 'Color', 'r');

         % Flash scanning beam
         beam = plot([0 x_max], [0 y_max], 'r', 'LineWidth', 3);
         drawnow;
         delete(beam);
      end
   end
end
```

6. System Output

The final output is a radar-like graphical display showing detected objects within a 180-degree scanning range. Objects appear in real-time at their corresponding angle and distance from the sensor.

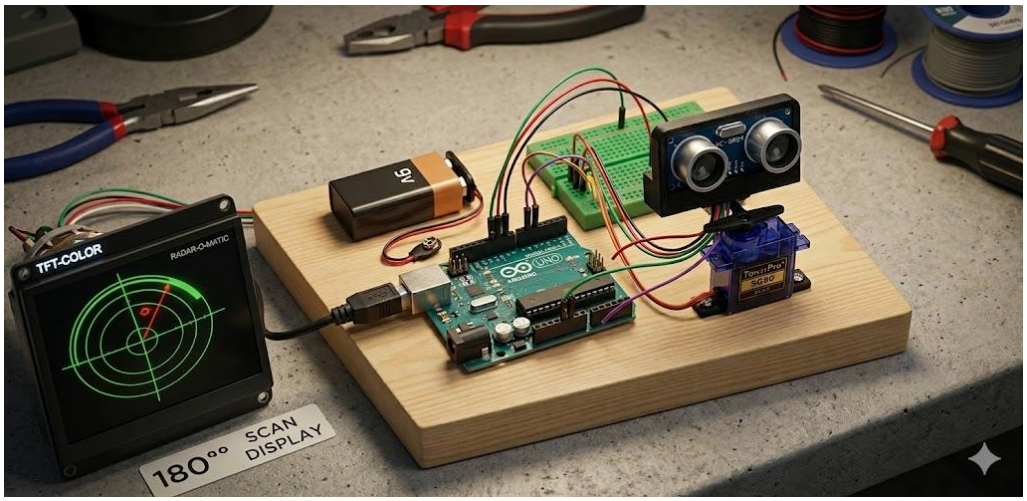


Figure -3 The 3D connection

7. Conclusion

This project demonstrates the integration of embedded systems and data visualization tools to build a functional radar prototype. The system effectively measures distance using ultrasonic sensing and provides real-time graphical representation using MATLAB, making it suitable for educational and prototyping applications.

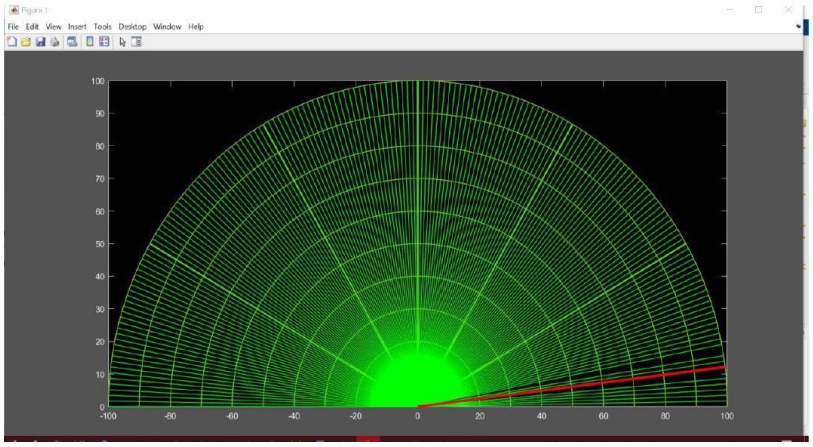


Figure-4 the Final Result